

```

64 var x = o.left;
65 var y = o.top;
66
67 var ax = settings.accX;
68 var ay = settings.accY;
69 var th = t.height();
70 var wh = w.height();
71 var tw = t.width();
72 var ww = w.width();
73
74 if (y + th + ay >= b &&
75     y <= b + wh + ay &&
76     x + tw + ax >= a &&
77     x <= a + ww + ax) {
78     //trigger the custom event
79     if (!t.appeared) t.trigger('appear', settings.data);
80
81     } else {
82     //it scrolled out of view
83     t.appeared = false;
84     }
85 };
86
87 //create a modified fn with some additional logic
88 var modifiedFn = function() {
89
90     //mark the element as visible
91     t.appeared = true;
92
93     //is this supposed to happen only once?
94     if (settings.one) {
95
96         //remove the check
97         w.unbind('scroll', check);
98         var i = $.inArray(check, $.fn.appear.checks);
99         if (i >= 0) $.fn.appear.checks.splice(i, 1);
100     }
101
102     //trigger the original fn
103     fn.apply(this, arguments);
104
105     //bind the modified fn to the element
106     //if (settings.one) t.one('appear', settings.data, modifiedFn);
107     //else t.on('appear', settings.data, modifiedFn);
108 };

```



MATERIA:

Programación Orientada a Objetos

Unidad 3 Programación Orientada a Objetos en Desarrollo Moderno

Docente:

Grupo:

Unidad #3 - Programación Orientada a Objetos en Desarrollo Moderno.

3. Introducción a Spring Boot y REST

3.1 ¿Qué es Spring Framework y qué aporta Spring Boot?

3.2 ¿Qué es una API REST? Principios y recursos.

3.3 Estructura de un proyecto Spring Boot.

3.4 Anotaciones básicas de Spring Boot

(`@SpringBootApplication`, `@RestController`,
`@RequestMapping`).

3.6 Creación del primer servicio REST (GET simple).

3.6.1 Uso de Lombok y Postman en el desarrollo moderno

3.1 ¿Qué es Spring Framework?

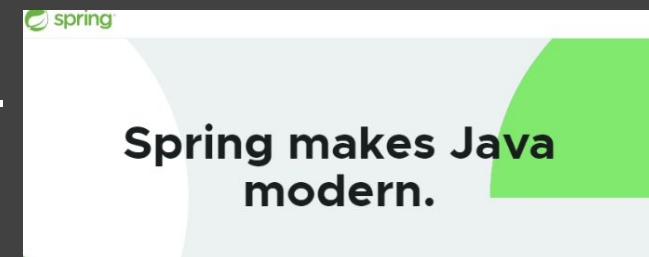
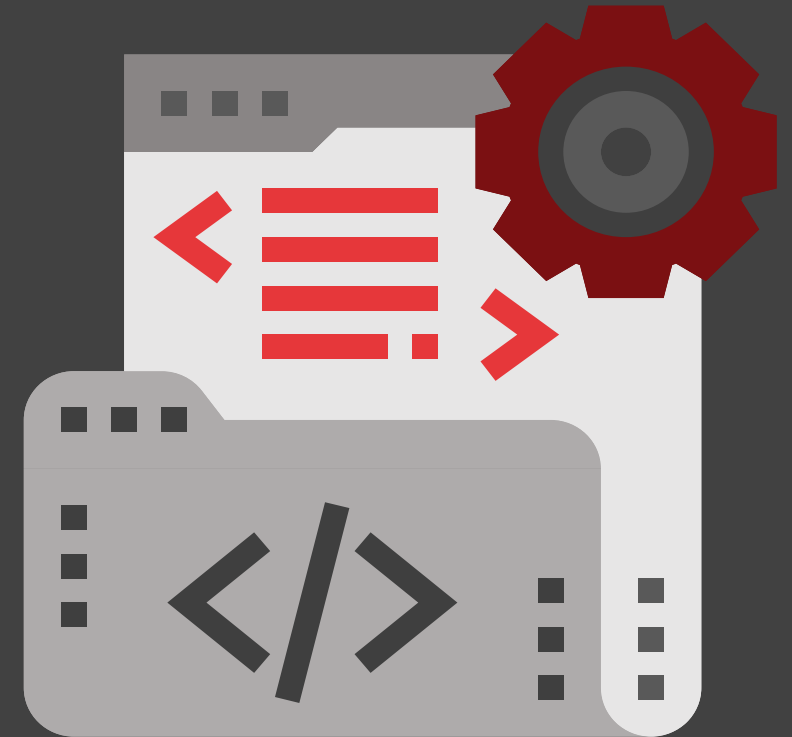
La "Tienda de Materiales" para Java

Un framework robusto y completo para construir aplicaciones Java empresariales.

Su núcleo es la Inversión de Control (IoC) y la Inyección de Dependencias (DI).

Ventaja: Es extremadamente potente y modular.

Desventaja: Requiere mucha configuración manual (antes en XML, ahora con clases Java).
como construir una casa desde los cimientos, pieza por pieza.



¿Qué aporta Spring Boot?

Spring Boot no reemplaza a Spring, ¡lo simplifica! Toma decisiones inteligentes por nosotros.

Sus Superpoderes:

- ✓ Autoconfiguración: Configura la aplicación automáticamente basándose en las librerías que incluyes.
- ✓ Servidor Web Embebido: Incluye un servidor (como Tomcat) dentro de la propia aplicación. ¡No necesitas instalar nada más!
- ✓ "Starters": Paquetes de dependencias que simplifican la configuración. Con spring-boot-starter-web tienes todo para empezar.

En resumen: Menos configuración, más desarrollo.

3.2 API REST - El Lenguaje de la Web

¿Qué es una API REST?

El "Menú de un Restaurante" Digital

API (Interfaz de Programación de Aplicaciones): Un contrato que permite que dos sistemas hablen entre sí.

REST (REpresentational State Transfer): Un estilo arquitectónico para diseñar esas APIs de forma estándar y predecible.

Principios Clave:

- Cliente-Servidor: La interfaz y el almacenamiento están separados.
- Sin Estado (Stateless): Cada petición es independiente. El servidor no recuerda nada de la petición anterior.

GET /productos/12: "Quiero VER el producto con ID 12"
DELETE /productos/12: "Quiero CANCELAR el producto con ID 12"



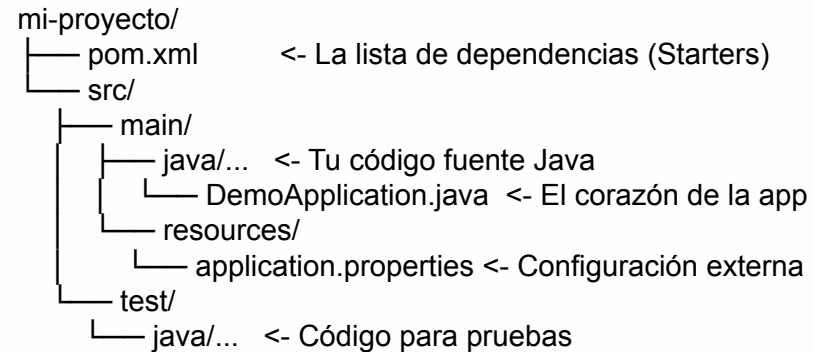
Los Componentes de REST

Componente	Analogía (Menú)	Ejemplo Técnico
Recurso (Sustantivo)	El plato que quieres.	/productos/usuarios/{id}
Los Componentes de REST Verbo HTTP (Acción)	Lo que quieres hacer con el plato.	GET, POST, PUT, DELETE
Representación	Cómo te entregan el plato.	JSON

3.3 Anatomía de un Proyecto

La Estructura de Carpetas

Un proyecto generado con Spring Initializr se ve así:



- **pom.xml**: El "inventario" de tu proyecto.
- **DemoApplication.java**: La clase con el método `main` que inicia todo.
- **application.properties**: Para configurar el puerto del servidor, la base de datos, etc.

3.4 Anotaciones Básicas de Spring Boot

Anotaciones - La Magia de Spring

Dándole Instrucciones a Spring

Las anotaciones son "etiquetas" (@) que le dicen a Spring qué hacer con nuestro código.

- `@SpringBootApplication`
 - Punto de entrada. Activa la autoconfiguración y el escaneo de componentes. Se coloca en la clase `main`.
- `@RestController`
 - Marca una clase como un **Controlador Web**. Su trabajo es recibir peticiones HTTP y devolver datos (normalmente JSON).
- `@RequestMapping("/api")`
 - Define una **ruta base** para todos los métodos de un controlador.
- `@GetMapping("/hola")`
 - Mapea una petición **GET** a un método específico. Existen equivalentes como `@PostMapping`, `@PutMapping`, etc.

Conectando Componentes con @Autowired

Anotaciones Clave: Conectando Componentes con @Autowired

El Corazón de la Inyección de Dependencias (DI)

@Autowired le pide a Spring que automáticamente "inyecte" una instancia de un componente (un "bean") dentro de otro.

¡Evita que tengas que escribir `new MiServicio()` en tu controlador!

Analogía del Chef 🧑🍳

- **El Controlador (El Chef):** Declara qué necesita: "Necesito un servicio de productos".
- **Spring (El Asistente Mágico ✨):** Escucha la petición, crea la instancia del servicio y se la entrega lista al chef.



Lombok - Escribe Menos, Haz Más

¿Qué es?

Una librería que genera automáticamente el código repetitivo de Java (getters, setters, constructores, etc.) usando anotaciones.

¿El Problema?

Las clases de modelo (POJOs) son muy largas y difíciles de leer.

Postman - Tu Navaja Suiza para Probar APIs

¿**Qué es?** Una aplicación gráfica para enviar cualquier tipo de petición HTTP y analizar las respuestas. Es la herramienta estándar para probar APIs.

¿**El Problema?** Un navegador web solo puede hacer peticiones **GET** fácilmente. No puedes probar **POST** o **PUT**, ni enviar datos en formato JSON.

Con Postman Puedes:

- ☒ Enviar peticiones **GET**, **POST**, **PUT**, **DELETE**, etc.
- ☒ Adjuntar un **cuerpo (Body)** en formato JSON para crear o actualizar recursos.
- ☒ Configurar **cabeceras (Headers)** para autenticación y otros metadatos.
- ☒ Ver la respuesta completa: **Código de Estado** (200 OK, 404 Not Found), el **cuerpo** de la respuesta y las **cabeceras**.

(Imagen de la interfaz de Postman mostrando una petición GET a <http://localhost:8080/api/products> y la respuesta JSON en la parte inferior.)

Esencial para verificar que todos tus endpoints funcionan como esperas.